MIT 8103 Advanced Database Systems – Portfolio Documentation

Advanced Database Systems Portfolio for **Finesse Klass** · First Semester CA, 2026/2027 Session · Master of Information Technology, Miva Open University

Student	Uchegbu Phinehas Kosisor · Student ID 301919344
GitHub repository	https://github.com/PhinehasdFalken/MIT-8013_Practical_Database_Portfolio (public repository)
Tools	PostgreSQL 16, MongoDB 7, Redis 7, Docker Compose, Python 3 (psycopg, pymongo, redis, matplotlib), Graphviz

1. Selected organisation (case study)

Finesse Klass is my Nigerian shoe brand. It sells ready-to-wear and **bespoke (made-to-measure)** shoes; most orders begin as Instagram direct messages and are delivered nationwide. All records in the portfolio are simulated with a deterministic generator (no real customer data).

Item	Description
Nature of the system	Online fashion retail: 6 categories, 120 styles sold as 2,400 size/colour variants with limited stock per size; workshop production of bespoke orders from foot measurements.
Database users	Owner/DBA; customer-care staff converting DMs into orders; workshop staff; dispatch staff; finance (payments, failed attempts, reconciliation); customers (checkout, order history).
Major data	Customers and addresses (state → region), categories, products, variants (SKU, price, stock), orders and order lines, bespoke measurements, payments, deliveries, stock-movement audit trail. Scale: 40,000 customers, 250,000 orders, 341,643 order lines, 264,082 payments.
Key operations	Place an order and take stock without overselling the last pair; record payments and retries; capture measurements; dispatch and track; customer order history; open-orders dashboard; monthly revenue by category; best-sellers; low-stock alerts.

2. The five portfolio activities

Activity	What was done (course concepts applied)	Key evidence / result
P1 Design & modelling	Requirements → conceptual (Chen ER: entities, attributes, composite and derived attributes, 1:N, M:N, 1:0..1) → logical (13 tables). Normalised from the Instagram order sheet UNF → 1NF → 2NF → 3NF → BCNF with all functional dependencies; keys, CHECK rules, composite FKs for business rules.	ER + relational diagrams; normalisation.md; 11 constraint tests, all rejected by the DBMS (entity, referential, domain, business-rule integrity).
P2 Query processing & optimisation	6-query workload; EXPLAIN (ANALYZE, BUFFERS) before/after physical design; B+ tree, multi-column, covering, hash and clustered organisation; query rewriting, memory (work_mem), materialised view, statistics/selectivity; cost-based optimisation.	Q1 ≈ 180–280× and Q4 ≈ 500–800× faster (2,942 → 2 buffers); materialised report thousands of times faster; index rejected for a 22-page table; clustering trade-off measured (25 → 4 vs 26 → 1,255 buffers).
P3 Transactions & concurrency	ACID mapped to evidence: COMMIT of a 6-step order, automatic and explicit ROLLBACK, SAVEPOINT, crash-recovery (WAL replay). Two-session demo of the 'last pair' lost update and three controls: SELECT…FOR UPDATE, REPEATABLE READ with retry, atomic conditional UPDATE; dirty/non-repeatable reads; deadlock and lock-ordering fix.	Naive version sells 1 pair twice; each control sells it once; lock waits proved via pg_stat_activity; deadlock detected and resolved; committed order survives an immediate shutdown.
P4 NoSQL & advanced models	Same data migrated to a MongoDB document model (embedded variants/items, extended references, flexible category attributes, $jsonSchema validation, aggregation pipelines, multikey indexes, atomic single-document stock update) and a Redis key-value model (carts with TTL, atomic drop-day counter, cache-aside).	Revenue identical to SQL (e.g. Boots Jan-2026 ₦142,588,000); COLLSCAN 250,002 docs → IXSCAN 24 docs; 8 buyers / 5 pairs → exactly 5 sold; suitability table (polyglot persistence).
P5 Distributed & cloud	4-node Docker cluster: streaming replication, delayed replica, async vs sync replication under a simulated partition (CAP), failover by promotion; horizontal fragmentation with the North fragment on a remote site (allocation, transparency, pruning, network cost); vertical fragmentation with per-fragment access; cloud/NDPA deployment discussion.	Stale read for 15.2 s then convergence; async commits continue during partition (AP), sync commit waits (CP); fragments complete/disjoint/reconstructable; 66,273 → 3 rows shipped with partition-wise aggregation.

3. Selected evidence

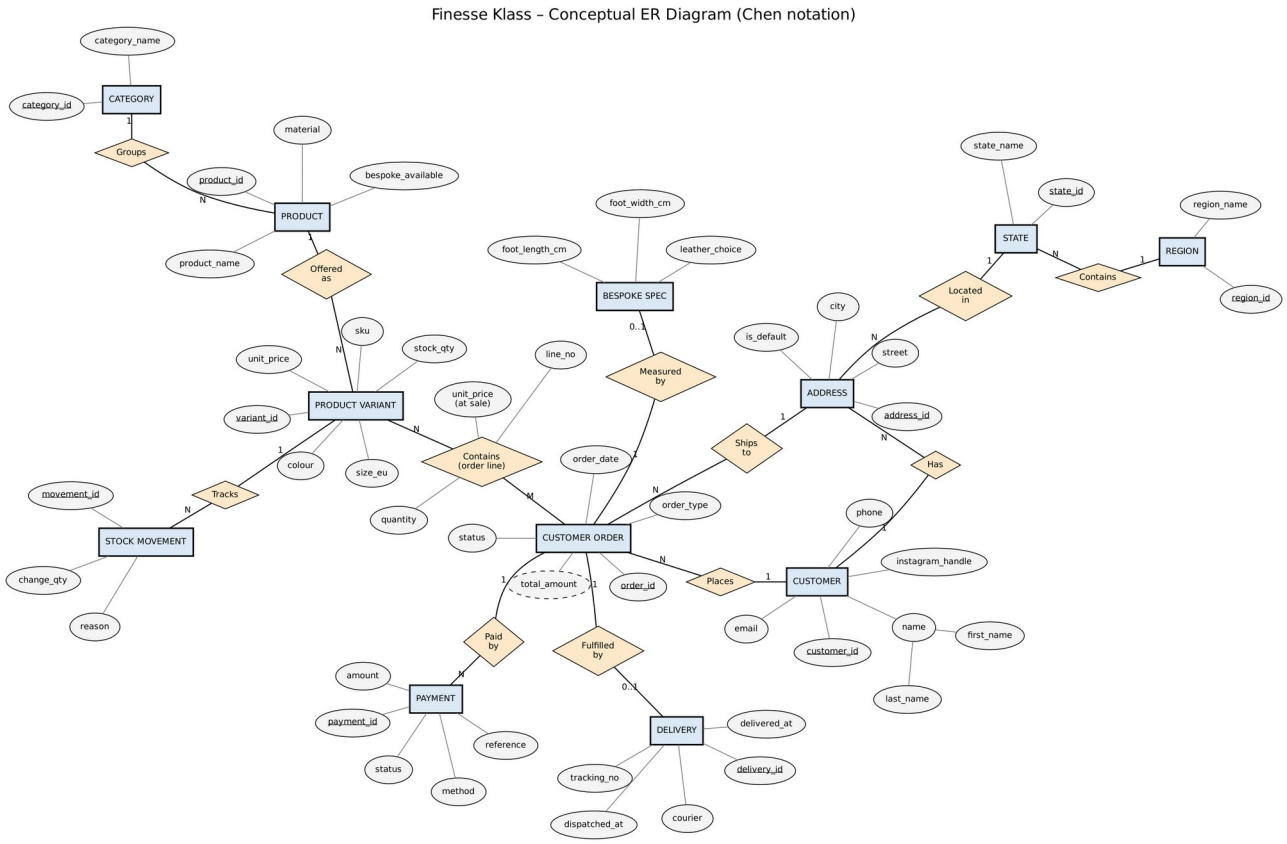


Figure 1 – Conceptual ER diagram of Finesse Klass (Chen notation), Portfolio 1

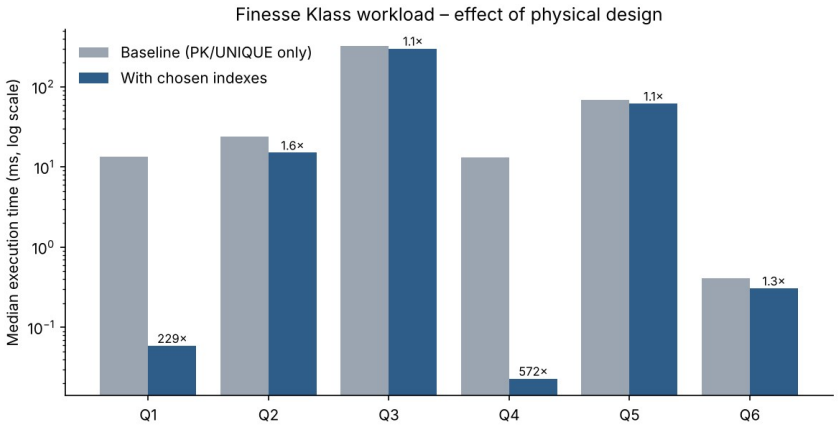


Figure 2 – Median execution time of the six workload queries before/after physical design, Portfolio 2 (log scale)

Every activity's folder contains the scripts, a README explaining how it was implemented and tested, the full outputs (execution plans, two-session timelines, explain() output, replication logs) in `evidence/`, and screenshots of the same runs in `evidence/screenshots/`.

4. Individual technical reflection

Design choices. Choosing my own brand made the requirements concrete: stock belongs to a size of a style, so product and variant were separated and a CHECK (stock ≥ 0) became the last line of defence. Two composite foreign keys enforce rules I did not want to leave to application code (delivery only to the customer's own address; measurements only on bespoke orders). The stored order total is a deliberate, controlled exception to normalisation, written only by the order transaction.

Lessons from measurement. Indexes transformed selective lookups but barely changed reports that read a large share of the data; one index saved no I/O at all and would only slow every sale. Rewriting a subquery as a JOIN by hand gained nothing because PostgreSQL had already flattened it, while NOT EXISTS was about twice as fast as NOT IN — optimisation must be measured, not assumed. A 2,596-row result estimated at about 80 rows showed how correlated columns defeat per-column statistics, and clustering showed that a table has only one physical order.

Challenges. Reproducing a race condition reliably required driving two sessions step by step with an observer proving the lock wait. CAP became practical when the same cluster behaved as AP with asynchronous and CP with synchronous replication; I therefore chose synchronous replication for orders and payments and asynchronous/delayed replicas for

catalogue reads. The document model simplified reads but re-introduced redundancy and lacks single-step atomicity across documents, so money stays in the ACID relational store. As a backend developer I will now read EXPLAIN before adding indexes, update stock only with atomic conditional statements, and choose the data model per workload.

5. AI use declaration

Item	Declaration
Tool	Claude (Anthropic), used as a coding and writing assistant.
Used for	Drafting first versions of the SQL, Python, MongoDB and Docker scripts and of the README/documentation text, from my instructions and the MIT 8103 course material; explaining execution-plan and replication output during analysis.
Review & verification	I ran every script on my own machine (PostgreSQL 16; MongoDB, Redis and the 4-node cluster in Docker); evidence files and screenshots come from those runs. I checked explanations against the course material, updated figures to my own outputs, made the design decisions, and can explain all submitted work.

6. References

Miva Open University (2025). MIT 8103 Advanced Database Systems [Course material], Study Sessions 1–6.

Elmasri, R. & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson.

Silberschatz, A., Korth, H. F. & Sudarshan, S. (2020). Database System Concepts (7th ed.). McGraw-Hill Education.

Özsu, M. T. & Valduriez, P. (2020). Principles of Distributed Database Systems (4th ed.). Springer.

Brewer, E. A. (2012). CAP twelve years later: How the "rules" have changed. Computer, 45(2), 23–29.

PostgreSQL Global Development Group (2024). PostgreSQL 16 Documentation: Query planning.

Nigeria Data Protection Commission (2023). Nigeria Data Protection Act (NDPA) 2023.